Reflection and meta-programming

A fragment based design

Document #:	D3435R0
Date:	2024-10-14
Project:	Programming Language C++
Audience:	Language Evolution

Reply-to:	Jean-Baptiste VALLON HOARAU <jbvallon@codereckons.com> Joel FALCOU <jfalcou@codereckons.com>

Contents

§ 1 Introduction

We present a practical, use-cases driven design for compile time reflection and meta-programming that has been implemented as a fork of the Clang compiler. Reflection is powered by a set of `consteval` functions providing access to properties of the language, while meta-programming is done by substitution of code fragments (not unlike Andrew Sutton's P2237, although with some differences).

The set of core use-cases that have informed the design includes : operators generation, generation of interfaces for other languages (e.g. Python), class registration, serialisation, implementation of arithmetic types, overload set and member "lifting", dynamic traits, a replacement for the `assert` macro. Ultimately, we think that meta-programming for C++ should aim to replace preprocessor macros and custom code generators that are common in existing C++ code.

This proposal is deliberately minimal and soft-edged. Minimal because we are proposing on the reflection side what is just enough to fullfil our meta-programming needs. Soft-edged because we are open to additions or reductions to the set of meta-programming use cases that we present here.

§ 2 Comparison with other designs

§ 2.1 P2996 : Reflection for C++26

This design differs from [P2996R5] mostly in that it is structured around 3 core types : `decl`, `expr`, and `type`, with additional types used to represent properties of those 3 core types (`identifier`, `name`, `source_location`, `template_argument`, etc.). P2996 argues that using types in the reflection API is a bad idea because we would then be unable to accomodate language evolution. We contend, however, that the categories of language properties we base the reflection API upon have been stable through the history of the language, and that there is no reason to believe that a change large enough to break it should happen. We contend that in general, weakly typed API are harder to use and more error prone.

We propose the ability to reflect on and construct expressions, which P2996 do not. This is necessary in virtually all of our use-cases.

Another difference is that P2996 proposes using `std::vector` for returning ranges of properties from a reflection value. Instead, our API uses the compiler builtins types `expr_list`, `type_list`, `decl_list` (etc.), mostly because it was easier to implement this way. But it turns out it has some advantages : first, these types can be literals and even *structurals*, so we can use them as template arguments. This is a major advantage over using `std::vector`, at least until C++ has a way to expand the set of structural type to include dynamic containers, and it's particularly important in our meta-programming design to have some dynamic containers that are structural (see section 4.7). Secondly, it's faster : all operations done in the interpreter with `std::vector` are done at native speed with a compiler builtin type. Finally, it frees the reflection API from a dependency to the standard library.

Otherwise, the reflection functionalities proposed here are essentially the same.

P2996 does not really propose a meta-programming mechanism beyond `template for`, so we cannot compare the twos in that aspect.

§ 2.2 P2237 : Meta-programming (with fragments)

The meta-programming design put forward by Andrew Sutton in [P2237R0] is the closest to ours, and it should come with no surprise that it has been an inspiration.

The most important difference is that our code fragments are parametrised with explicit captures, removing the need for the "unquote operator". We also do not propose consteval blocks. Furthermore, our injection model is based on "code builders", which allows to inject in a context and provides information about it.

P2237 touches upon the issue of using context provided names in code fragments. In practice, we've found that this problem can always be solved through passing down

reflection of the declarations needed to the injection code, or, when that proves to be too verbose, performing lookup in the context of injection. We will talk about this in more detail in the meta-programming section.

§ 2.3 P3294 : Code Injection with Token Sequences

[P3294R1] proposes a meta-programming model based on tokens injection. While this is viable (see : Rust), the strength of tokens based meta-programming, which is the ability to construct any kind of code without syntactic or semantic verification, is also its weakness. It works when it works, when it doesn't, woe betides the user.

Fragments based meta-programming obviously guarantee syntactical correctness, but also allows the compiler to perform semantic verification at each step of program construction.

P3294 points out that too much semantic checking at the point of fragment declaration limits their usefulness, in particular with respect to name lookup. We agree : this is why we are in favor of two-phase lookup, and why we do not propose anything like the "required declarations" of P2237.

Admittedly, our prototype cannot yet implement the LoggingVector use case put forth by P3294. We will explore a possible solution in the last part of the use-case section.

§ 2.4 Our previous work

We've previously talked about an AST-based meta-programming model in a series of blog posts. What we are presenting here is very different – we are abandoning the previous model as a serious contender for standardisation. While it worked, it was very cumbersome to use.

We also considerably reduced the reflection API surface. While we've kept the core types `decl`, `expr`, and `type`, we do not propose refined types anymore, nor do we propose statements traversal, expressions traversal, and querying function bodies (hence the `stmt` type is gone).

While some of that might proves to be desirable in the future, our focus is on providing a replacement for preprocessor macros and custom code generators, and the aforementioned features are not necessary to achieve this goal.

§ 2.5 Rust : macros and `syn`

Rust macros are essentially functions receiving a sequence of tokens as input, and producing another sequence of tokens to be injected. In Rust terminology, derive macros and attributes-like macros are macros who accept *Rust code* as input, and produce Rust code as output. This is interesting, because since Rust does not have reflection, most users rely on the crate `syn` to parse the input into an AST.

The `syn` crate is of interest to us, as it gives an example of what kind of reflection API is useful in practice. The `syn` API (like our previous design) is based on 4 sum types : `Expr`, `Type`, `Stmt` and `Item` (roughly equivalent to our `decl`). Of course, since `syn` is not part of Rust standard library, the challenge of accommodating language evolution is not the same. However, it is remarkable that the `syn` API had little breaking changes over the years, relatively to the speed at which Rust evolved.

Another point of interest is how `syn` is used : it's a *direct* dependency in 6% of the crates on crates.io (the Rust package manager), but it's a *transitive* dependency of 59% of them! In other words, a large proportion of code relies on comparatively few tools written with it (it's even used in the rustc compiler), but most of Rust code does not use it directly.

We expect meta-programming in C++ to be used in a similar manner : a tool with which is built few but widely used software components, be they code-generating functionalities (e.g. operators generation, language bindings generation) or "plain" generic functionalities (e.g. automatic serialisation/traversal, aggregate formatting, enum-to-string).

To get back to the claim made in P2996 that changing a typed reflection API would be impossible (from the amount of work needed to update its dependents), we think that the way `syn` is used and has evolved paints a different picture.

§ 3 Reflection

The reflection operator `^` transform a construct of the language into a value. The operand of the reflection operator can be a type, expression, declaration, or the global namespace. The syntax of a *reflection-expression* is as follows :

```
reflection-expression :
  ^ ( expression )
  ^ type-id
  ^ id-expression
  ^ nested-name-specifier_opt namespace-name
  ^ nested-name-specifier_opt template-name
  ^ ::
```

The type of the reflection expression is determined by its operand. The result can be either `std::meta::type`, `std::meta::expr`, `std::meta::decl`, or `std::meta::overload_set` (which is a range of `decl`).

Example :

```
int x, y;
std::meta::expr e = ^(x + y);
std::meta::type t = ^int[101];
std::meta::decl d = ^x;
std::meta::overload_set os = ^operator<; // a range of std::meta::decl
```

Furthermore, the equality operator is defined for all reflection types. Its semantics will be defined in the following sections.

§ 3.1 Types

A reflected type is a value of type `std::meta::type`. Types are equality comparable. Implementations are free to preserve aliases (and perhaps required to do so to a reasonable degree), but equality comparison shall compare the canonical types.

```
using MyInt = int;
static_assert( ^MyInt == ^int );
```

§ 3.1.1 Basic type manipulation

The `std::meta` namespace provides a set of functions which cover the functionalities of `<type_traits>`. These functions have the same name and behavior as their template counterpart. Obviously, some of the traits in `<type_traits>` like `is_same` or `enable_if` are not needed.

```
// std::meta
consteval bool is_integral(type t);
consteval bool is_floating_point(type t);
consteval bool is_array(type t);
// ...
consteval bool is_constructible(type t, type_list tl);
consteval bool is_copy_constructible(type t);
// ...
consteval type remove_reference(type t);
consteval type add_lvalue_reference(type t);
consteval type remove_pointer(type t);
consteval type add_pointer(type t);
// ...
consteval type decay(type t);
consteval type remove_cv_ref(type t);
// ...
```

§ 3.1.2 Template type queries

A pretty common need is to inspect the content of template types. For example, we often want to constrains a function to only accept instance of a template. The function `is_instance_of(type t, decl d) -> bool` helps us do that :

```
template <class... Ts>
  requires ((is_instance_of(remove_reference(^Ts), ^tuple)) && ...)
auto tuple_cat(Ts&&... tpls);
```

We can use the function `template_of(type t) -> decl` to the same end :

```
template <class T, int N>
  requires (template_of(remove_reference(^T)) == ^tuple)
auto&& get(T&& tpl);
```

We can also query template arguments, which is a necessary facility to the implementation of `tuple_cat` :

```
using namespace std::meta;

consteval auto tuple_cat_result_type(type_list tl) {
  template_argument_list args;
  for (auto t : tl)
    for (auto a : template_arguments_of(t))
      push_back(args, a);
  return substitute(^tuple, args);
}
```

Here the function `substitute` expand the argument list into `tuple`. An evaluation error is produced if substitution fails. However, the instantiation of the definition is done upon use.

§ 3.1.3 Conversion to declaration

Some types, like enumeration, class, and class template types have a corresponding declaration, which can be accessed through `decl_of` :

```
consteval bool is_or_contains_pointer(type t) {
  if (is_pointer(t))
    return true;
  if (!is_class(t))
    return false;
  for (auto f : fields(decl_of(t)))
    if (is_pointer(type_of(f)))
      return true;
  return false;
}
```

(we will talk about the function `fields` in the next chapter)

§ 3.1.4 Hashing

We propose that a `hash` function shall be defined for types, but we're not quite sure if the value produced should be stable across invocations of the compiler. In our current implementation, it isn't.

§ 3.1.5 Comparison to P2996

Here is the `tuple_cat_result_type` function written with P2996 :

```
consteval                                                  std::meta::info
        tuple_cat_result_type(std::vector<std::meta::info> tl) {
  std::vector<std::meta::info> args;
  for (auto t : tl)
    args.append_range(template_arguments_of(t));
  return substitute(^tuple, args);
}
```

We've already talked about the pros and cons of using compiler builtins types vs `std::vector`, but clearly, having the `std::vector` interface is an ergonomic advantage

over the rather spartan interface that our builtins types provide, and it would be the ideal solution if `std::vector` could be structural.

§ 3.2 Declarations

We propose a set of queries related to declarations that should be enough to power most meta-programming applications. Those are the functionalities over declarations we needed to achieve our use-cases :

— query their name

— query their members + (convenience functions for instance data members/functions…)

— query their kind (class/function/namespace/data member…)

— query their type

— if classes, query their bases classes

— lookup a member

— if function, query their parameters

— if enumerator, query their underlying values

— query their default value (for data member, function parameter)

Those functionalities are covered by the following functions :

```
// name
consteval name name_of(decl d);
consteval identifier identifier_of(decl d);

// members
consteval decl_list children(decl d);
consteval field_list fields(decl d); // the instance data members of d, if
        d is a class, in a random-access range
consteval decl_list functions(decl d);
consteval decl_list enumerators(decl d);

// kind
consteval bool is_function(decl d);
consteval bool is_class(decl d);
consteval bool is_namespace(decl d);
consteval bool is_variable(decl d);
consteval bool is_enum(decl d);
// ... ect

consteval type type_of(decl d);
consteval base_class_list bases_of(decl d);

// lookup a member
consteval decl_list lookup(decl d, name n); // (this lookup looks at
        members of base classes if d is a class)
consteval decl_list lookup_excluding_bases(decl d, name n);
```

```
consteval decl_list parameters_of(decl d);
consteval decl_list template_parameters_of(decl d);

consteval expr initializer(decl d); // initializer if variable, default
        value if data member or parameter
consteval expr underlying_value(decl d); // underlying value if enum
        constant
```

We propose that template instances can be `decl`s and shall be represented as template specialisations declarations – so that their data members can be reflected just like a class, for example.

There are some questions surrounding declarations reflection that we haven't fully resolved yet : we are not quite sure how to handle using-directives or using-declarations, whether or not implicitly declared members should be visible on traversal, or if querying the name of an anonymous declaration should produce an evaluation error or an empty identifier.

§ 3.2.1 Declaration name

The function `name_of` returns a `std::meta::name`, unlike P2996 which returns a `std::string_view`. Returning a string_view is fine for the uses that P2996 demonstrate. In our case, it's a bit awkward, especially when dealing with operators and conversion names : we might want to inject an operator which depends on the name of a function (see meta-programming : expressions), or obtains the type of the conversion name, which we can't do from a string view. We also don't want to have to parse the string ourselves to know which kind of name we have.

In the proposed API, a `std::meta::name` can be an identifier, an operator, a conversion type, or the name of a special member function. We have a separate type to represents identifiers, `std::meta::identifier`. Furthermore, an enumeration type `std::meta::operator_kind` is provided to represents operators.

We consider the name of special member functions to be class independent, that is, `name_of(^Struct1::Struct1) == name_of(^Struct2::Struct2)`.

A set of predicates over `std::meta::name` is provided :

```
// namespace std::meta
consteval bool is_identifier(name n);
consteval bool is_operator(name n);
consteval bool is_conversion(name n);
consteval bool is_constructor(name n);
consteval bool is_destructor(name n);
```

To retrieve the identifier, operator, or conversion type, `name` can be cast to its subtypes :

```
name n = ...;
auto op = static_cast<operator_kind>(n);
```

```
auto id = static_cast<identifier>(n);
auto ty = static_cast<type>(n);
```

The evaluation of these casts errors out if the name is not an operator, identifier or type conversion, respectively.

§ 3.2.2 Object parameter

Should the object parameter, even if implicit, be included in the range returned by the `parameters_of` query? We're inclined to say yes (we treat the implicit object parameter as an anonymous declaration). At the very least, the answer should not change depending on its explicitness, which would be weird. But there are cases where we are only interested in the "inner" parameters, and to that end we provide a function `inner_parameters_of`.

§ 3.3 Expressions

For now we do not propose any queries related to expressions beyond `type_of`, `location`, and maybe a `hash` function (see enum-to-string use case).

Equality comparison of expressions test the equivalence of two expressions. Parens within the expression are ignored.

```
static_assert( ^(1 + 2) == ^((1 + 2)) );
static_assert( ^(1 + 2) != ^(2 + 1) );
```

For convenience, a reflection of a list of comma separated expressions gives an `expr_list` (instead of a relatively uninteresting comma-expression).

```
expr_list e = ^(1, 2, 3);

consteval expr tuple_cat_gen(expr_list el);

template <class... Ts>
constexpr auto tuple_cat(Ts... tpls) {
  constexpr auto result = tuple_cat_gen( ^(tpls...) ); // also construct
        an expr_list
}
```

Some functions which construct expressions will be covered in the meta-programming part.

§ 3.4 Conversion to text and diagnosis

Conversion to text is often needed by meta-programs, and it has been a necessity in a good proportion of our use cases. To this end we propose a general purpose mechanism to build chain of characters from builtins and reflection typed values : `std::meta::stringstream`. It behaves essentially like `std::stringstream` :

```
std::meta::stringstream os;
os << "hello " << ^(1 + 2) << " " << ^std::meta << " " << ^int[121] << " "
        << name_of(^os);
// the content of the stream is now "hello 1 + 2 std::meta int[121] os"
```

Its content can be used in three ways. First, `begin` and `end` yield iterators over the content of the stream, returning a `char` pr-value.

Second, the function `make_literal_expr` takes a stream as argument to construct a string literal expression (see use cases enum-to-string and class registration).

Finally, it can be used to emit diagnostics. Emitting user-friendly diagnostics is needed as we expect a lot of meta-programming code to come with pre-conditions, which, if not fulfilled, will result in failure at the point of injection.

The following intrinsics can be used to emit diagnostics :

```
template <class T>
concept DiagMsg = ^T == ^std::meta::stringstream || ^T == ^const char*;

consteval void ensure(bool b); // compile-time assertion
consteval void ensure(source_location loc, bool b); // ditto, but emit an
        error at loc

consteval void error(const DiagMsg auto& s);
consteval void error(source_location loc, const DiagMsg auto& s);
consteval void warning(const DiagMsg auto& s);
consteval void warning(source_location loc, const DiagMsg auto& s);
```

Furthermore, we propose to constexpr-ise `std::format`, and that the `<format>` header provides the following functions :

```
// namespace std::meta
template <class Str, class Args>
consteval void error(Str&& str, Args&&... args);
// implemented as
// error( std::format(str, static_cast<Args&&>(args)...).c_str() );

template <class Str, class Args>
consteval void error(source_location loc, Str&& str, Args&&... args);
template <class Str, class Args>
consteval void warning(Str&& str, Args&&... args);
template <class Str, class Args>
consteval void warning(source_location loc, Str&& str, Args&&... args);
```

A specialisation of `std::formatter` exists for all reflection types (which is quite straightforward to implement with `std::meta::stringstream`).

§ 3.5 Syntax issues : the caret and Objective-C

As noted by P3381, using the caret as the reflection operator causes a conflict with the following Objective-C syntax :

```
type-id(^ident)();
```

which can be parsed as either :

— a variable named ident holding a Obj-C block returning type-id and taking no arguments, and

— a cast of a reflection of `ident` into a type-id and then a call of operator()

The authors therefore propose to change ^ to ^^.

While this is a problem, we contend that the conflict could be worked around. We think that this syntax pattern has a rather low chance of being used for reflection purpose, and that few people would be negatively impacted by a compiler interpreting this pattern as an Objective-C declaration, especially if they deliberately compile with Objective-C blocks extension enabled (and if an user intends the second meaning, this expression can be easily rewritten non-ambiguously).

Furthermore, our prototype already use the token ^^ for the “eager” reflection operator (a reflection operator who is never dependent, i.e. can be used to reflect on dependent entities prior to template substitution). This is a rather advanced topic that we won’t talk about here, but it’s another motivation for us to try to find a compromise and keep ^.

The second issue pointed by P3381 is not a problem for us (see the next section).

§ 4 Meta-programming

To begin with the syntax, we’ve chosen a prefix % as the injection operator.

It can be used to inject types and expressions, or to introduce an injection statement.

Injection statements inject *code fragments*. Fragments are pieces of code that can be parametrised by explicitly captured values. Those captured values are akin to template parameters : they are constant expressions when referenced from inside the fragment.

Substitution of the fragment is performed *on injection*. The captures values are initialised when evaluating the fragment expression, and destroyed after injection is done.

Code fragments are used with *builders*. A builder is an handle to a language entity being constructed, it comes from an injection statement, which can appear at function, class, namespace, or enum scope.

An injection statement looks like this :

```
%inject_something();
```

It can appears, at function, class, namespace or enumeration scope. If the injection operand is a call-expression, the callee receives a reference to a *builder* as first argument.

This builder is typed according to the scope enclosing the injection statement (either `function_builder`, `class_builder`, `namespace_builder`, or `enum_builder`).

A builder can be used to inject code via `operator<<`, the right-hand-side must be suitably typed (see below). It also gives access to the declaration behind constructed via the `decl_of` function, and the injection location via `location`.

If the type of the injection operand is not void, it will be injected (and thus must be convertible to the fragment type associated with the context).

Injections statements can also appears in fragments, in which case they will be evaluated upon injection of the fragment.

The syntax of fragments and parametrised expression is as follows :

```
parametrised-expression :
  ^ [captures] ( expression )

function-fragment :
  ^ [captures] { statement }

class-fragment :
  ^ [captures]opt struct { member-specification }

namespace-fragment :
  ^ [captures]opt namespace { namespace-body }

enum-fragment :
  ^ [captures]opt enum { enumerator-list }
```

The types of these expressions are `expr`, `function_fragment`, `class_fragment`, `namespace_fragment`, and `enum_fragment`, respectively.

As previously stated, expressions and types can also be injected. The syntax

```
% constant-expression
```

inject a type or expression depending on the context in which it appears. The type of the operand must be `type` or `expr`. In the declarator context, disambiguation is required to introduce an injected type. The following syntax must be used to inject a type in a function prototype, variable or data member declarator.

```
% typename ( constant-expression )
```

§ 4.1 Expressions

The primary way to construct expressions is using parametrised expressions. We don't call them fragments, because unlike fragments, substitution is performed when the *parametrised-expression* expression is encountered.

For example :

```
consteval expr make_add(expr l, expr r) {
  return ^ [l, r] (%l + %r);
}

consteval expr lift(overload_set os) {
  return ^ [os] ( [] (auto&&... args) { return (%os)(args...); } );
}
```

There is a few things that plain substitution of expression does not do well, at least not without further extension of the language. For example, the operators generation use case depends on being able to construct an operator expression from a `std::meta::operator_kind`.

We've been thinking of implementing operator injection with the following syntax :

```
consteval expr make_operator_expr(operator_kind op, expr l, expr r) {
  return ^[op, l, r] ( %operator(op)(%l, %r) );
}
```

But for now, we are simply providing a `make_operator_expr` function which does exactly that.

§ 4.1.1 Expression pack injection

Another thing that substitution needs is the ability to expand ranges. For example, given a range of expressions, we want to be able to expand it into an argument list to make a call expression :

```
consteval expr make_call_expr(expr callee, expr_list args) {
  return ^[callee, args] ( (%callee)(%...args...);
}
```

The syntax

```
% ... constant-expression
```

creates a *pack-injection expression*. This expression contains an unexpanded parameter pack whose elements results from the constant evaluation of the injection operand, which in this case must be convertible to `expr_list`. It behaves just like a parameter pack would, and can be used along template parameters pack, in fold expressions, etc.

So in the above example, `%...args` creates an unexpanded pack, and the following ellipsis expand it into the argument list.

Pack injection can happen with a non-dependent operand, in which case the expansion is done immediately :

```
constexpr auto args = ^(lots, of, arguments);
return foo(%...args..., 1, 2) + bar(%...args...);
```

§ 4.1.2 Member injection expression

The syntax

```
expression .%( constant-expression )
```

creates a *member injection expression*. The injection operand must be convertible to either `decl` or `name`.

For example, here is an implementation of `get` for tuple, assuming data members named `mX` :

```
template <int N, class... Ts>
auto& get(tuple<Ts...>& t) { return t.%(cat("m", N)); }
```

§ 4.2 Function fragments

Function fragments are a sequence of statements, optionally parametrised by captures values. Within a function fragment, the return type is treated as dependent. Statements whose validity depends on the presence of an enclosing scope, such as break, continue, or case statements, can be used – an error will be emitted upon injection if their use is invalid in the injection scope.

The capture list must appear even if empty, to disambiguate against Objective-C blocks.

To illustrate, here is an implementation of an enum-to-string function :

```
consteval void gen_enum_to_string(function_builder& b, decl Enum)
{
  for (auto enumerator : children(Enum))
  {
    std::meta::stringstream ss;
    ss << name_of(enumerator);
    b << ^[enumerator, lit = make_literal_expr(ss)] {
      case %enumerator : return %lit;
    };
  }
}

template <class E>
  requires (is_enum(^E))
constexpr const char* to_string(E e) {
  switch(e) {
    %gen_enum_to_string(^E);
  }
}
```

§ 4.3 Injection of names

To generate an arbitrarily named declaration, we must be able to inject names. To this end, we propose that the syntax

```
% name ( constant-expression )
```

shall produce an injected name, which can appear in the place of a *declarator-id*, *class-name*, *enum-name*, or *enumerator*.

The injection operand must be of the type `std::meta::name`.

If the injected name is illegal for the declaration for which it is intended (e.g. if it is an operator name for a variable declaration), an error is emitted. Furthermore, constructors cannot be declared with an injected name.

Note that injected names are for *declaration purpose only*, they cannot be used for looking up an existing entity.

§ 4.4 Namespace fragments

Namespace fragments are a sequence of namespace members potentially parametrised by captures values.

```
consteval void inject_stuff(namespace_builder& b, int k) {
  b << ^ [k] namespace {
    void f();
    void %name(cat("f", k)) ();
  };
}
```

In this example, the injection statement `%inject_stuff(101)` will produce the functions `f` and `f101`.

§ 4.5 Enum fragments

Enum fragments are a sequence of enumerators (or injections statements) potentially parametrised by captures values.

As an example, here is a simple function which generates a sequence of enumerators suitable for using the enum as a sum of flags :

```
consteval void gen_flagsum(enum_builder& b, const std::vector<identifier>&
        ids) {
  int k = 1;
  for (auto id : ids)
  {
    b << ^ [id, k] enum { %name(id) = k };
    k *= 2;
  }
}

enum MyEnum {
  %gen_flagsum({"a", "b", "c"})
};
```

§ 4.6 Class fragments

A class fragment is a sequence of member declarations potentially parametrised by capture values. In a class fragment, the type of `this` is dependent. This naturally allows for two-phase lookup at the point of injection :

```
static constexpr auto postfix_increment = ^ struct
{
  auto operator++(int) {
    auto tmp = *this;
    ++tmp;
    return tmp;
  }
};
```

Andrew Sutton in P2237 touches upon the problem of declaring constructors within a fragment or referencing the class in which we will be injecting. We propose the same solution as Sutton : class fragments can be named, and that a reference to this name shall be substituted upon injection.

```
consteval auto inject_stuff(class_builder& b) {
  return ^ struct T {
    // declaring constructors/destructors
    T() {...}
    ~T() {...}
    // referencing the destination type
    bool operator==(const T& o) const { ... }
  };
}
```

The injection of the member functions bodies is done only when the class in which the injection is performed has been completed. This forbids using captures that contains references to local values in the bodies of member functions, as the evaluation context at that point will be gone.

```
template <class R>
consteval expr gen_eq(expr l, expr r, R&& members) {
  expr res = ^(true);
  for (auto m : members)
    res = ^ [res, m, l, r] ( res && ((%l).%(m) == (%r).%(m)) );
  return res;
}

consteval   auto   eq_according_to(class_builder&   b,   std::vector<decl>
        members) {
  std::span<const decl> members_span {members.begin(), members.end()}; //
        naively trying to avoid a copy...
  b << ^ [members_span] struct T {
    bool operator==(const T& o) const {
      %gen_eq(^(*this), ^(o), members_span);
    }
  };
}
```

In this example, the evaluation of the injection operand in `operator==` will errors out as we attempt to read out-of-lifetime values. The solution is to simply capture the vector itself.

§ 4.7 Fragment captures : technicals considerations

What kind of values can be used as fragments captures? This is a complex question.

It depends on *how* the capture is used. Some use will requires it to be substituted by a perennial, equivalent expression as in

```
auto e = ^ [k = 101] (some_function(k));
```

while others does not, as in :

```
std::vector<std::meta::decl> vec = ...;
auto e = ^ [vec] ( %make_some_expr(vec) );
```

The latter simply reference `vec` in the computation of the inner injection operand, but once that is done, the expression obtained is that which is produced by `make_some_expr`, so there is no need to produce an expression equivalent to `vec`. We will call the first kind of capture use *perennial use*, and the second kind *instant use*.

So what kind of capture can be perennially used? At the very least, only those of literal types, because a perennial use of a capture of class type imply declaring a `constexpr` global to reference in the substituted expression – and some might want to say : only structural types, because we want to "unique" them into a set so that we don't emit more declarations than we need. Either way, this precludes the usual dynamic containers.

This is why we've kept the compiler builtins types for dynamic containers : since they are structural (because the elements are immutable), they can always be used as fragment captures, and as template arguments.

Class fragments impose yet another constraint. Because the body of their member functions will be substituted only once the class is complete, even an *instant use* of a capture can be problematic if said capture references local values.

§ 4.7.1 Captures and template definition

The problem of replacing a non-structural value by a perennial expression is why we cannot, for example, generate the body of a function template from a `std::vector` at the moment :

```
consteval void gen_foo(const std::vector<int>& vec, expr fn);

consteval auto inject_stuff(std::vector<int> vec) {
  return ^ [vec] namespace
  {
    template <class T>
    auto foo(T Fn) {
```

```
        %gen_foo(vec, ^(Fn)); // compiler error : instant use fragment
        capture cannot be part of dependent injection operand
    }
  };
}

%inject_stuff({1, 2, 3});
```

The injection operand `gen_foo(vec, ^(Fn))` is still value-dependent after fragment injection, and cannot be evaluated. But we cannot have an expression equivalent to `vec` outside of the fragment injection context, so the compiler emits an error.

What can be done about this? We've been exploring what we call "eager reflection", which allow to reflect on entities prior to template substitution, and would let us generate the body of template using non-structural values. With eager reflection, `^^(Fn)` would the reflection of `Fn` prior to template substitution, and thus would be non-dependent, so the injection operand can be evaluated on fragment injection.

§ 5 Use-cases

Here is a series of use cases which guided our design. We will make comparisons to P2996 when applicable.

§ 5.1 Enum to string

We've already shown a simple implementation of enum to string in the function fragments section. However, a complete implementation need to take into account repeating values, or enumeration used as a sum of flag. P2996 does the former implicitly by generating a chain of `if`. We could also do that, but for the sake of exploration, we would like to implement our function as a single switch.

```
consteval void gen_enum_to_string(function_builder& b, decl d)
{
  constexpr_map<expr, expr> map;

  for (auto e : children(d))
  {
                                map.try_emplace(        underlying_value(d),
        make_literal_expr(identifier_of(d)) );
  }

  for (auto m : map)
  {
    b << ^ [m] {
      case %m.first : return %m.second;
    };
  }
}

template <class T>
  requires is_enum(^T)
```

```
std::string_view enum_to_string(T val) {
  switch(std::to_underlying(val)) {
    %gen_enum_to_string(^T);
    default : return "<invalid>";
  }
}
```

This works, with the caveat that `expr` is hashable.

§ 5.1.1 Bitsum enum-to-string

Here is the implementation which handle the sum of flags case. A generic enum-to-string function should pick this one if all the enumeration members are power of twos.

```
consteval void gen_enum_bitsum_to_string(function_builder& b, decl Enum,
        expr val, expr res, expr tail)
{
  for (auto e : children(Enum))
  {
    b << ^ [val, res, tail, e]
    {
      if (%val & %e) {
        if (%tail)
          %res += " | ";
        %res += %make_literal_expr(identifier_of(e));
        %tail = true;
      }
    };
  }
}

template <class T>
constexpr std::string enum_bitsum_to_string(T val) {
  std::string res;
  bool tail = false;
  %gen_enum_bitsum_to_string(^T, ^(val), ^(res), ^(tail));
  return res;
}
```

A similar function implemented with P2996, assuming `template for`, would be essentially the same :

```
template <class T>
constexpr std::string enum_bitsum_to_string(T val) {
  std::string res;
  bool tail = false;
  template for (constexpr auto e : enumerators(^T))
  {
    if ( [:e:] & val )
    {
      if (tail)
        res += " | ";
      res += name_of(e);
      tail = true;
    }
```

```
  }
  return res;
}
```

§ 5.2 Parsing command-line options

Here we show how to implement a command-line arguments parser. It's essentially the same code as P2996, except that `template for` is replaced by repeated injection within an injection statement. Here again, we have to pass down the reflection of the local values we use in our injection statement, which is not needed with `template for`.

```
template <class T>
consteval expr to_string_lit_expr(T val) {
  std::meta::stringstream ss;
  ss << val;
  return make_literal_expr(ss);
}

consteval void gen_parse_args(function_builder& b, expr opts, expr args) {
  auto Opts = decl_of(type_of(opts));
  for (auto f : fields(Opts))
  {
    b << ^ [opts, args, f]
    {
      auto it = std::ranges::find_if(%args,
        [](std::string_view arg){
                    return  arg.starts_with("--")  &&  arg.substr(2)  ==
        %to_string_lit_expr(identifier_of(f));
      });

      if (it == (%args).end()) {
        // no option provided, use default
        continue;
      } else if (it + 1 == (%args).end()) {
        std::print(stderr, "Option {} is missing a value\n", *it);
        std::exit(EXIT_FAILURE);
      }

      using T = %typename(type_of(f));

      auto iss = std::ispanstream(it[1]);
      if (iss >> (%opts).%(f); !iss) {
         std::print(stderr, "Failed to parse option {} into a {}\n", *it,
        %to_string_lit_expr(^T));
        std::exit(EXIT_FAILURE);
      }
    };
  }
}

template <class Opts>
constexpr auto parse_args(std::span<std::string_view> args)
{
  parse_result<Opts> opts;
```

```
  %gen_parse_args(^(opts), ^(args));
  return opts;
}
```

§ 5.3 Tuple

Since our model allows to generate code within declarations, we can inject all the fields of tuple directly, and keep it a simple aggregate. The function `expand_as_fields` creates the members `m0`…`mN`. `apply` and `tuple_cat` are both implemented by the function `expand_fields`, which performs an expansion for each element of an expression list.

```
consteval void expand_as_fields(class_builder& b, type_list tl)
{
  int k = 0;
  for (auto t : tl)
    b << ^ [t, p = k++] struct { %typename(t) %name(cat("m", p)); };
}

consteval void collect_fields(std::meta::expr_list& el, std::meta::expr e)
{
  using namespace std::meta;
  auto cd = decl_of(remove_reference(type_of(e)));
  for (auto fd : fields(cd))
    push_back(el, ^ [e, fd] ((%e).%(fd)));
}

consteval std::meta::expr_list expand_fields(std::meta::expr_list el)
{
  using namespace std::meta;
  expr_list res;
  for (expr e : el)
    collect_fields(res, e);
  return res;
}

consteval  std::meta::expr  get_by_type(std::meta::type  T,  std::meta::expr
        E)
{
  auto cd = decl_of(remove_reference(type_of(E)));
  for (auto fd : fields(cd))
    if (type_of(fd) == T)
      return ^ [E, fd] ( (%E).%(fd) );
  std::meta::stringstream os;
  os << "type " << T << " not contained in " << cd;
  error(os);
  return ^(0);
}

template <class... Ts>
struct tuple
{
  constexpr bool operator<=>(const tuple<Ts...>& ts) const = default;

  %expand_as_fields(type_list{^Ts...});
```

```
};

template <unsigned Index, class Tpl>
  requires ( std::meta::is_instance_of(^Tpl, ^tuple) )
constexpr auto&& get(Tpl&& tpl)
{
  return tpl.%(fields(^Tpl)[Index]);
}

template <class T, class Tpl>
  requires ( std::meta::is_instance_of(^Tpl, ^tuple) )
constexpr auto&& get(Tpl&& tpl)
{
  return %get_by_type(^Tpl, ^(tpl));
}

namespace impl
{
  consteval type tuple_cat_result_type(type_list tl)
  {
    template_argument_list args;
    for (auto t : tl)
      for (auto a : template_arguments_of(t))
        push_back(args, a);
    return substitute(^tuple, args);
  }
}

template <class... Ts>
  requires ( std::meta::is_instance_of(^Ts, ^tuple) && ... )
constexpr auto tuple_cat(Ts&&... ts)
{
            using      ResultType      =      %impl::tuple_cat_result_type(
        {remove_reference(^Ts)...} );
  return ResultType{ %...expand_fields(^(ts...))... };
}

template <class Fn, class... Ts>
  requires ( std::meta::is_instance_of(^Ts, ^tuple) && ... )
constexpr decltype(auto) apply(Fn fn, Ts&&... ts)
{
  return fn( %...expand_fields(^(ts...))... );
}
```

§ 5.3.1 Comparison with P2296

P2996 cannot inject code within a declaration, therefore their implementation of tuple cannot be a simple aggregate and must declare a constructor, which is both an ergonomic and performance loss. Otherwise, the implementation of `get` is the same.

Here is their implementation of `tuple_cat`, thanks to Tomasz Kaminski :

```
template<std::pair<std::size_t, std::size_t>... indices>
struct Indexer {
   template<typename Tuples>
```

```
   // Can use tuple indexing instead of tuple of tuples
   auto operator()(Tuples&& tuples) const {
     using ResultType = std::tuple<
       std::tuple_element_t<
         indices.second,
                   std::remove_cvref_t<std::tuple_element_t<indices.first,
         std::remove_cvref_t<Tuples>>>
       >...
     >;
        return  ResultType(std::get<indices.second>(std::get<indices.first>
        (std::forward<Tuples>(tuples)))...);
   }
};

template <class T>
consteval auto subst_by_value(std::meta::info tmpl, std::vector<T> args)
    -> std::meta::info
{
    std::vector<std::meta::info> a2;
    for (T x : args) {
        a2.push_back(std::meta::reflect_value(x));
    }

    return substitute(tmpl, a2);
}

consteval auto make_indexer(std::vector<std::size_t> sizes)
    -> std::meta::info
{
    std::vector<std::pair<int, int>> args;

    for (std::size_t tidx = 0; tidx < sizes.size(); ++tidx) {
        for (std::size_t eidx = 0; eidx < sizes[tidx]; ++eidx) {
            args.push_back({tidx, eidx});
        }
    }

    return subst_by_value(^Indexer, args);
}

template<typename... Tuples>
auto my_tuple_cat(Tuples&&... tuples) {
                                    constexpr         typename          [:
        make_indexer({type_tuple_size(type_remove_cvref(^Tuples))...})  :]
        indexer;
              return    indexer(std::forward_as_tuple(std::forward<Tuples>
        (tuples)...));
}
```

It still has to rely on rather elaborate template meta-programming, but P2296 helps in the construction of a pack of pair of indices.

§ 5.4 Variant

Here we reuse the `expand_as_fields` function defined above to generate the variant storage. This code is pretty much the same as the implementation of P2296, except for the storage and `visit` (which they do not implement – it could be done with `template for`, though it would result in a chain of `if`).

```
template <int N>
struct constant {
  static constexpr auto value = N;
};

consteval void gen_visit(function_builder& b, expr data, expr vis)
{
  auto d = decl_of(type_of(data));
  int k = 0;
  for (auto f : fields(d))
  {
    b << ^ [data, vis, p = k++]
    {
      case p : return (%vis)( (%data).%(f) );
    };
  }
}

consteval void gen_visit_with_index(function_builder& b, expr data, expr
        vis)
{
  auto d = decl_of(type_of(data));
  int k = 0;
  for (auto f : fields(d))
  {
    b << ^ [data, vis, p = k++]
    {
      case p : return (%vis)( (%data).%(f), constant<p>{} );
    };
  }
}

template <class... Ts>
struct variant : variant_base
{
  static constexpr bool trivial_dtor = (is_trivially_destructible(^Ts) &&
        ...);
  using ctor_selector = impl::overload_selector<Ts...>;

      static consteval type alternative(unsigned idx) { return
        type_list{^Ts...}[idx]; }
     static consteval unsigned index_of(type T) { return
        impl::find_first_pos(type_list{^Ts...}, T); }

  template <class... Args>
  constexpr variant(Args&&... args)
   requires ((sizeof...(Args) > 0) && requires { impl::ctor_selector{}
        ({(Args&&)args...}); })
  : data{}
  {
```

```
      constexpr auto emplace_idx = decltype( impl::ctor_selector{}
      ({(Args&&)args...}) )::value;
  data.template emplace<emplace_idx>( (Args&&) args... );
  index_m = emplace_idx;
}

constexpr variant(const variant& v)
: data{}
{
  emplace_from(v);
}

constexpr variant(variant&& v)
  noexcept ((is_nothrow_move_constructible(^Ts) && ...))
  requires (is_move_constructible(^Ts) && ...)
: data{}
{
  emplace_from((variant&&) v);
}

template <unsigned N, class... Args>
constexpr void emplace(Args&&... args) {
  destroy();
  data.template emplace<N>( (Args&&) args... );
  index_m = N;
}

constexpr variant& operator=(const variant& o)
  requires (is_assignable(^Ts) && ...)
{
  destroy();
  emplace_from(o);
}

constexpr auto index() const {
  return index_m;
}

union Data
{
  constexpr Data()
  : xxx{}
  {}

  template <unsigned N, class... Args>
  constexpr Data(emplace_at_index<N>, Args&&... args)
  {
    this->emplace<N>((Args&&)args...);
  }

  template <unsigned N, class... Args>
  constexpr void emplace(Args&&... args) {
    std::construct_at( &this->%(cat("m", N)), (Args&&)args... );
  }

  constexpr ~Data()
```

```
    requires (not trivial_dtor)
  {
  }

  constexpr ~Data()
    requires (trivial_dtor)
  = default;

  impl::empty_t xxx;
  %expand_as_fields(type_list{^Ts...});
};

constexpr ~variant()
  requires (not trivial_dtor)
{
  visit( impl::destruct_element{}, *this );
}

constexpr ~variant()
  requires (trivial_dtor)
= default;

template <unsigned Index>
constexpr auto& get(this auto&& self) {
  return self.data.%(fields(^Data)[Index + 1]);
}

template <class T>
constexpr T& get(this auto&& self) { return self.get<index_of(^T)>(); }

template <class F>
constexpr decltype(auto) visit(this auto&& self, F&& fn) {
  switch(index_m) {
    %gen_visit(^(self.data), ^(fn));
  }
}

template <class F>
constexpr decltype(auto) visit_with_index(this auto&& self, F&& fn) {
  switch(index_m) {
    %gen_visit_with_index(^(self.data), ^(fn));
  }
}

Data data;

private :

template <unsigned Idx, class... Args>
constexpr void emplace_no_reset(Args&&... args) {
  data.template emplace<Idx>( (Args&&)args... );
}

template <class V>
constexpr void emplace_from(V&& o)
```

```
  {
    this->visit_with_index( [] (auto& elem, auto idx) {
      index_m = idx.value;
      this->emplace_no_reset<idx.value>( elem );
    }, (V&&) o);
  }

  constexpr void destroy() {
    if constexpr ( not trivial_dtor )
      visit( impl::destruct_element{}, *this );
  }

  unsigned char index_m;
};
```

§ 5.5 Universal formatter

This is our equivalent implementation of the universal formatter proposed in P2996. Here again, because we do not have `template for`, we must pass down the reflection of our local values to the injection function. Otherwise, the code is similar.

```
consteval expr to_string_lit(type T) {
  std::meta::stringstream os;
  os << T;
  return make_literal_expr(os);
}

struct universal_formatter {
  constexpr auto parse(auto& ctx) { return ctx.begin(); }

  template <typename T>
  auto format(T const& t, auto& ctx) const {
    auto out = std::format_to(ctx.out(), "{}{{", to_string_lit(^T));

    auto delim = [first=true]() mutable {
      if (!first) {
        *out++ = ',';
        *out++ = ' ';
      }
      first = false;
    };

    % [] (function_builder& b, expr out, expr val, expr delim)
    {
      for (auto base : bases_of(^T))
      {
        b << ^ [out, val, delim, base] {
          %delim;
          %out = std::format_to(%out, "{}", static_cast< %base >(%val));
        };
      }
    } (^(out), ^(t), ^(delim()));

    % [] (function_builder& b, expr out, expr val, expr delim)
```

```
    {
      for (auto f : fields(^T))
      {
        b << ^ [out, val, delim, f] {
          %delim;
          %out = std::format_to(%out, "{}", (%val).%(f));
        };
      }
    } (^(out), ^(t), ^(delim()));

    *out++ = '}';
    return out;
  }
};
```

§ 5.6 Operators generation

Probably our most compelling use-case so far is operators generation. The way we achieve this is by delegating the implementation to a member function `apply_op` taking an `operator_kind` as template argument. Within the body of this member function, the user can construct an operator expression generically from the template argument (using `make_operator_expr`, or operator injection).

```
consteval void declare_op(class_builder& b, type operand_type,
        operator_kind op)
{
  auto this_ty = type_of(b);

  if (!std::meta::is_compound_assign(op))
    this_ty = add_const(this_ty);
  this_ty = add_lvalue_reference(this_ty);

  b << ^ [this_ty, operand_type, op] struct
  {
      constexpr decltype(auto) %name(op) (this %typename(this_ty) self,
        %typename(operand_type) rhs) {
      return self.template apply_op<op>(rhs);
    }
  };
}

consteval void declare_operators(class_builder& b, type operand_type,
        std::span<operator_kind> ops)
{
  for (auto op : ops)
    declare_op(b, operand_type, op);
}

consteval void declare_arithmetic(class_builder& b, type operand_type)
{
    // assuming math_compound and math_op are arrays containing the
        appropriate operator_kind
  declare_operators(b, operand_type, math_compound);
  declare_operators(b, operand_type, math_op);
```

```
}

// usage...

template <class T, unsigned N>
struct vec
{
  template <operator_kind Op>
    requires (std::meta::is_compound_assign(Op))
  constexpr auto& apply_op(const vec<T, N>& o)
  {
    for (int k = 0; k < N; ++k)
      (%make_operator_expr(Op, ^(data[k]), ^(o.data[k])));
    return *this;
  }

  template <operator_kind Op>
    requires (not std::meta::is_compound_assign(Op))
  constexpr auto apply_op(const vec<T, N>& o) const
  {
    auto Res {*this};
        (%  make_operator_expr(std::meta::compound_equivalent(Op),  ^(Res),
        ^(o)) );
    return Res;
  }

  constexpr bool operator==(const vec<T, N>& o) const {
    for (int k = 0; k < N; ++k)
      if (data[k] != o.data[k])
        return false;
    return true;
  }

  constexpr bool operator!=(const vec<T, N>& o) const {
    return !(*this == o);
  }

  T data[N];

  %declare_arithmetic( ^const vec& );
};
```

Here we are using predicates over the kind of operator to pick the correct overload.

§ 5.7 Class registration

Here is a simple utility to register a class constructor in a map. The registrar must be a type along the lines of :

```
template <class Key, class Base>
class basic_class_register
{
  using Ctor = Base*(*)();
  std::unordered_map<Key, Ctor> ctor_map;
```

```
  public :

  using base_t = Base;

  template <class T>
  void emplace(Key key) {
    ctor_map.emplace( key, +[] () -> Base* { return new T{}; } );
  }

  Base* create(Key key) {
    auto it = ctor_map.find(key);
    if (it == ctor_map.end())
      return nullptr;
    return it->second();
  }
};

struct Base {
  virtual ~Base() {}
};

inline static basic_class_register<std::string, Base> registrar;
```

Then, a class declaration inheriting from Base can be registered via :

```
%register_class(^registrar, ^Derived);
```

We can also register the classes contained in a namespace or a class via :

```
namespace ns {
  struct A : Base {};
  struct B : Base {};
}

%register_classes(^registrar, ^ns);
```

The implementation is fairly simple : we inject a static variable with a placeholder name so that the class registration is executed at the beginning of the program. By default, we use the (qualified) name of the class as key.

```
consteval void register_class(namespace_builder& b, decl registrar, type
        cd, expr key)
{
  b << ^ [cd, registrar, key] namespace
  {
    static bool _ = ((%registrar).template emplace< %typename(cd) >(%key),
        true);
  };
}

consteval void register_class(namespace_builder& b, decl registrar, type
        cd)
{
  std::meta::stringstream os;
```

```
  os << cd;
  register_class(b, registrar, cd, make_literal_expr(os));
}
```

The `register_classes` function traverse a namespace or class and register all the classes that inherit from `base_t` declared by the registrar :

```
consteval void register_classes(namespace_builder& b, decl registrar, decl
        source)
{
  auto base = type_of(*begin(lookup(registrar, "base_t")));

  ensure(is_namespace(source) || is_class(source), "source must a class or
        namespace");

  for (auto d : children(source))
  {
    if (is_class(d) && is_derived_from(type_of(d), base))
      register_class(b, registrar, d);
  }
}
```

§ 5.8 Enum flagsum generation

```
%declare_flagsum("MyFlagSum", {"a", "b", "c"});

// Produce the code :
enum class MyFlagSum {
  a = 1, b = 2, c = 4
};

constexpr MyFlagSum operator|(MyFlagSum a, MyFlagSum b) {
  return (MyFlagSum) (std::to_underlying(a) | std::to_underlying(b));
}
```

This is an interesting use case, because we want to inject a declaration (operator |) which uses the name of the enum we just injected. Because we have no mechanism yet to do this, we first inject the enumeration, then use `lookup` to retrieve it, and perform a second injection for the operator.

```
consteval     void     gen_flagsum_constants(enum_builder&     b,     const
        std::vector<identifier>& ids)
{
  int k = 1;
  for (auto id : ids)
  {
    b << ^ enum [id, k] { %name(id) = k };
    k *= 2;
  }
}

consteval  void  declare_flagsum(namespace_builder&  b,  identifier  Name,
        std::vector<identifier> ids)
{
```

```
  b << ^ [&ids] namespace
  {
    enum class %name(Name)
    {
      %gen_flagsum_constants(ids)
    };
  };

  auto Enum = *begin(lookup(decl_of(b), Name));
  auto EnumTy = type_of(Enum);

  b << ^[EnumTy] namespace {
         constexpr  %typename(EnumTy)  operator|  (%typename(EnumTy)  A,
        %typename(EnumTy) B) {
                  return  (%typename(EnumTy))  (std::to_underlying(A)  |
        std::to_underlying(B));
    }
  };
}
```

§ 5.9 Members and overload set lifting

Encapsulating members and overload sets in a callable is quite straightforward, as declaration names and overload set can be passed as template arguments. This would not be possible if names were `string_view`/if overload sets used `std::vector`.

```
template <overload_set OS>
struct lift_overload_set {
  template <class... Args>
  constexpr decltype(auto) operator()(Args&&... args) {
    return (%OS)(static_cast<Args&&>(args)...);
  }
};

consteval expr lift(overload_set os) {
  return ^ [os] ( lift_overload<os>{} );
}

template <name N>
struct lift_member {
  template <class... Args>
  constexpr decltype(auto) operator()(auto&& head, Args&&... args) {
    return head.%(N)(static_cast<Args&&>(args)...);
  }
};

consteval expr lift(name n) { // lift a member named n
  return ^ [n] ( lift_member<n>{} );
}
```

Usage :

```
std::vector<std::vector<int>> vecs = ...;
std::ranges::sort( vecs, %lift("size") );
```

§ 5.10 `ensure` : replacement for assert

Here we implement a simple, hygienic assert macro replacement, which is able to adapt to whether or not it is invoked in a consteval/constexpr function. We can emit an error at compile-time using `std::meta::error`.

```
consteval expr make_error_string(expr e, source_location loc, expr
        error_msg, bool PrintLoc = true) {
  std::meta::stringstream ss;
  if (PrintLoc)
    ss << loc << ": ";
  ss << "Assertion failed (" << e << ")";
  ss << " : " << error_msg << "\n";
  return make_literal_expr(ss);
}

consteval void gen_consteval_failure(function_builder& b, expr e,
        source_location loc, expr msg)
{
  b << ^ [e, loc, msg]
  {
    std::meta::error(loc, %make_error_string(e, loc, msg, false));
  };
}

consteval void gen_runtime_failure(function_builder& b, expr e,
        source_location loc, expr msg)
{
  b << ^ [e, loc, msg]
  {
    std::cerr << %make_error_string(e, loc, msg) << std::endl;
    std::abort();
  };
}

consteval void gen_failure(function_builder& b, source_location loc, expr
        e, expr msg)
{
  auto fn = decl_of(b);

  if (is_consteval(fn))
  {
    gen_consteval_failure(b, e, loc, msg);
  }
  else if (is_constexpr(fn))
  {
    b << ^ [msg, e, loc]
    {
      if consteval {
        % gen_consteval_failure(e, loc, msg);
      }
      else {
        % gen_runtime_failure(e, loc, msg);
      }
    };
```

```
  }
  else
    gen_runtime_failure(b, e, loc, msg);
}

consteval void ensure(function_builder& b, expr e, const char* message)
{
  b << ^ [e, msg = make_literal_expr(message), loc = location(b)]
  {
    if (!%e)
    {
      % impl::gen_failure(loc, e, msg);
    }
  };
}
```

§ 5.11 Cloning types : logged class

As P2237 points out (in section 7.2.6 : Injecting parameters), injecting functions or template parameters with fragments is a challenge.

In this section we explore a possible solution, which we have yet to fully implement.

Generating a function prototype with an arity determined from input values could be done with range expansions, which are needed in other places like expression construction anyway :

```
consteval auto inject_foo(type_list tl) {
  return ^ [tl] namespace {
    void foo( %...typename(tl)... params );
  };
}
```

But this doesn't let us specify default arguments, which is a big drawback for the purpose of cloning an interface.

One possible solution is injection statements within a function parameter list :

```
consteval void inject_params(function_parameters_builder&) {
  // placeholder syntax for function parameter list fragment
  b << ^(| int x |);
  b << ^(| float y = 1.2 |);
}

void foo( %|inject_params()| );
```

Here, the placeholder syntax `%| constant-expression |` produce an injection statement within a parameter list. Unfortunately, we need a different syntax than expression injection here, otherwise we have no way to know if this is a function declaration or a variable whose initialiser is injected.

Parameters injection goes hand in hand with the ability to retrieve the parameters in a context where we do not yet know their names. In this exploratory exemple, this is done by the expression `parameters_of(^this)`, where `^this` results in a reflection of the declaration in which it appears.

With those mechanisms, we can implement the `LoggingVector` class described in P3294 :

```
consteval type transpose_obj_type(type new_class, type old_obj_ty) {
  auto new_obj_ty = new_class;
  if (is_const(remove_reference(old_obj_ty)))
    new_obj_ty = add_const(new_obj_ty);
  if (is_lvalue_reference(old_obj_ty))
    new_obj_ty = add_lvalue_reference(new_obj_ty);
  else if (is_rvalue_reference(old_obj_ty))
    new_obj_ty = add_rvalue_reference(new_obj_ty);
  return new_obj_ty;
}

consteval void clone_parameters(function_parameters_builder& b, decl f,
        type new_class) {
  for (auto p : inner_parameters_of(f))
  {
    auto ty = type_of(p);
    if (remove_cv_ref(ty) == remove_cv_ref(object_type(f)))
      ty = transpose_obj_type(new_class, ty);
    if (!has_initializer(p))
      b << ^[p, ty] (| %typename(ty) %name(name_of(p)) |);
    else
       b << ^[p, ty] (| %typename(ty) %name(name_of(p)) = %initializer(p)
        |);
  }
}

consteval expr_list fwd_args(decl_list params) {
  expr_list res;
  for (auto p : params)
  {
    auto ty = add_rvalue_reference(type_of(p));
    push_back(res, ^[ty] (static_cast< %ty >(%p)) );
  }
  return res;
}

consteval expr to_string_lit(name n) {
  stringstream ss;
  ss << n;
  return make_literal_expr(ss);
}

consteval void clone_interface(class_builder& b, decl d) {
  for (auto f : functions(d))
  {
    if (is_static_member(f))
      continue;
```

```
    auto obj_ty = transpose_obj_type(type_of(b), object_type(f));

    b << ^ [f, obj_ty] struct T
    {
      %typename(return_type(f)) %name(name_of(f)) ( this %typename(obj_ty)
        self, %|clone_parameters(f, ^T)| ) {
        constexpr auto args_list = fwd_args(parameters_of(^this));
        std::println("Calling {}", %to_string_lit(name_of(f)));
        return self.impl.%(name_of(f))( %...args_list... );
      }
    };
  }
}
```

For comparison, here is the equivalent code of P3294 :

```
template <typename T>
class LoggingVector {
    std::vector<T> impl;

public:
    LoggingVector(std::vector<T> v) : impl(std::move(v)) { }

    consteval {
        for (std::meta::info fun : /* public, non-special member functions
        */) {
            auto argument_list = list_builder();
            for (size_t i = 0; i != parameters_of(fun).size(); ++i) {
                argument_list += ^{
                       // we could get the nth parameter's type (we can't
        splice
                      // the other function's parameters but we CAN query
        them)
                    // or we could just write decltype(p0)
                    static_cast<decltype(\id("p", i))&&>(\id("p", i))
                };
            }

            queue_injection(^{
                \tokens(make_decl_of(fun, "p")) {
                    std::println("Calling {}", \(name_of(fun)));
                    return impl.[:\(fun):]( [:\(argument_list):] );
                }
            });
        }
    }
};
```

It relies on a `make_decl_of` function which construct the token sequence necessary to clone a function. We could also pretend to have such a function (taking a function fragment as argument) but it must be pointed out that its implementation would be perhaps more difficult, as we cannot factor out declaration specifiers such as `virtual` or `constexpr`, which is trivial with tokens manipulation.

Nevertheless, we are able to map the old class type to the new one, which enables us to declare e.g. constructors or `swap`, which P3294 cannot do.

§ 6 Future work

In many of these exemples, we have to pass down reflection of local declarations to our meta-programming algorithms. This is where P2296 and P3294 have an ergonomic advantage. The first, because `template for` allows to refers to local declarations within a code generating statement (arguably, this is not a matter of model : our design could include `template for`). The second, because tokens sequences are unchecked, the declarations references will be resolved at the point of injection. More work is needed to find a solution to this problem – perhaps `template for` is enough.

Adjacent to this problem is better support for two phase lookup, and referring to declarations whose name is injected. Currently this is done manually (see enum flagsum example), but ultimately we would like to be able to refer to declarations via an injected name.

Finally, constructing template and functions parameters lists, as well as injecting constructors initialisers, is for now unsupported. This is a domain where tokens injection has an advantage, as it is allows complete freedom with little specification.

While this design and its implementation are still a work in progress, everything we have discussed in this paper, unless otherwise specified, has been implemented. We hope to show that a fragment based meta-programming model along a “reasonably typed” reflection API is a viable avenue for standard C++.

§ 7 References

[P2237R0] Andrew Sutton. 2020-10-15. Metaprogramming.
https://wg21.link/p2237r0

[P2996R5] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-08-14. Reflection for C++26.
https://wg21.link/p2996r5

[P3294R1] Barry Revzin, Andrei Alexandrescu, Daveed Vandevoorde. 2024-07-16. Code Injection with Token Sequences.
https://wg21.link/p3294r1